

QB: Chapter 2.2 — Stacks & Recursion

Unit 2 | 4×2 -mark | 5×5 -mark | 4×10 -mark | Total: 13 questions
Lectures: L22–L26 | BT Levels: BT1, BT2, BT3, BT5 | COs: CO2, CO3

Section A: 2-Mark Questions

Q1. [BT1 | CO2] Define a *stack*. State the principle it follows (LIFO). Name the two primary operations and describe what each does to the `top` pointer.

Q2. [BT1 | CO3] Define *postfix notation* (Reverse Polish Notation). Convert the infix expression $A + B * C$ to postfix without using a formal algorithm. Explain why postfix notation requires no parentheses.

Q3. [BT2 | CO2] Describe *stack overflow* and *stack underflow*. State the conditions under which each occurs in an array-based stack and in a linked-list-based stack. Why is overflow not possible in a linked-list stack (assuming unlimited memory)?

Q4. [BT2 | CO3] Explain the role of a stack in evaluating postfix expressions. What does the algorithm do when it encounters an operand? What does it do when it encounters an operator? Give a one-step example for each.

Section B: 5-Mark Questions

Q5. [BT3 | CO2] Implement an array-based stack with `push(item)`, `pop()`, `peek()`, and `is_empty()` operations (capacity = 5). Trace the following sequence: `push(10)`, `push(20)`, `push(30)`, `pop()`, `peek()`. For each operation, show the state of the stack array and the value of `top`. State the final contents of the stack.

Q6. [BT5 | CO2, CO3] Implement the balanced parenthesis checker using a stack. The algorithm must handle `(, ), {, }, [, ]`. Trace the algorithm on: (a) Input `"( { [ ] } )"` — show stack state after each character. (b) Input `"( { [ ) } )"` — identify the exact token at which mismatch is detected. Explain why a stack is the natural structure for this problem (relate to LIFO and nesting depth).

Q7. [BT5 | CO2, CO3] Evaluate the postfix expression $5\ 3\ 2\ * +\ 8\ 4\ / -$ using a stack. Show the stack state after processing each token in a table (token, action, stack contents). State the final result. Verify the result by writing the equivalent infix expression and computing it using standard operator precedence.

Q8. [BT5 | CO3] Convert the infix expression $A + B * C - D / E$ to postfix using the Shunting-Yard algorithm. Show the operator stack and output queue state after processing each token in a table. Apply correct precedence rules ($*$ and $/$ before $+$ and $-$). State the final

postfix expression.

Q9. [BT5 | CO3] Convert the infix expression $(A + B) * (C - D^E)$ to postfix. Trace the Shunting-Yard algorithm showing the operator stack and output queue after every token. Correctly handle: (a) nested parentheses, (b) right-associativity of the exponentiation operator $^$. State the final postfix expression.

Section C: 10-Mark Questions

Q10. [BT5 | CO2, CO3] (*Experiment E6 — Evaluate Reverse Polish Notation, LeetCode #150*)

The RPN Evaluator (LeetCode #150) must evaluate expressions given as a list of string tokens.

(a) Implement a complete RPN evaluator using a stack. Include handling of all four operators (+, −, ×, /) with integer truncation towards zero for division. Write the algorithm with clear error handling for division by zero and empty-stack access.

(b) Trace the evaluator on tokens ["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"]. Show the stack state after processing each token. State the final result.

(c) Explain why RPN eliminates the need for operator precedence rules and parentheses. Describe the complete pipeline from infix input to final numeric result: (i) infix-to-postfix conversion (Shunting-Yard), (ii) postfix evaluation (stack). Identify the time and space complexity of each stage.

Q11. [BT5 | CO2, CO3] (*Experiment E15 — Combination Sum, LeetCode #39*)

The Combination Sum problem (LeetCode #39) finds all unique combinations of numbers from a candidates array that sum to a target. Each number may be used multiple times.

(a) Design a backtracking recursive algorithm. Write the complete recursive function with: the function signature, base cases (target reached, target exceeded), the loop over candidates, the choose/explore/unchoose pattern, and the pruning condition that prevents re-examining smaller candidates.

(b) Trace the full backtracking tree for candidates = [2, 3, 6, 7], target = 7. Label each tree node with the remaining target. Clearly mark branches that are pruned and mark the leaves that yield valid combinations.

(c) Explain the relationship between recursion and the call stack. At the deepest point of the recursion tree traced in (b), describe the call stack frames (function, local variables, remaining candidates). Compare this implicit call stack with the explicit stack used in an iterative depth-first backtracking implementation.

Q12. [BT5 | CO2, CO3]

A compiler's expression parser must process the infix expression: $(A + B^2) * C - D / (E + F)$

(a) Apply the Shunting-Yard algorithm. Show, after every token, the state of the operator stack and output queue. Handle: left-to-right associativity for $+$, $-$, $*$, $/$ and right-to-left associativity for $^$. State the final postfix expression.

(b) Draw the expression tree for the original infix expression. Verify that an *inorder traversal* of the tree produces the original infix expression (with parentheses) and that a *postorder traversal* produces the postfix expression from part (a).

(c) Evaluate the postfix expression from (a) using a stack for values $A = 2$, $B = 3$, $C = 4$, $D = 8$, $E = 1$, $F = 3$. Show the stack state after every token is processed and state the final numerical result.

Q13. [BT5 | CO2, CO3]

The Tower of Hanoi puzzle requires moving n disks from peg A to peg C using peg B as auxiliary, without ever placing a larger disk on a smaller one.

(a) Write the recursive algorithm for Tower of Hanoi. Derive the recurrence relation $T(n) = 2T(n - 1) + 1$ by identifying the three sub-tasks. Solve the recurrence by unrolling to prove $T(n) = 2^n - 1$.

(b) Trace the complete sequence of moves for $n = 3$ disks. Verify that the total number of moves equals $2^3 - 1 = 7$. Present the moves in a numbered list showing (disk, from-peg, to-peg).

(c) Explain how the system call stack supports recursive execution. Draw the call stack state (listing active frames with their parameters) at the deepest recursion depth for $n = 3$. For $n = 64$, calculate the required number of moves and explain why this algorithm is computationally infeasible, relating it to stack depth and exponential time growth.